Données manquantes en pratique

Cas complets, IPCW et imputation multiple avec R

David Hajage

18 novembre 2026

Sommaire

Introduction **3**
En ligne ou via Rstudio ? . . . 3
Solutions . . . 3

1 Les données et leurs valeurs manquantes **5**
1.1 Présentation des données . . . 5
1.2 Combien de données manquantes ? . . . 6
1.3 Le mécanisme est-il MCAR ? . . . 7

2 Analyse sur cas complets **9**

3 Pondération IPCW **11**
3.1 Étape 1 : modéliser la probabilité d'être complet . . . 11
3.2 Étape 2 : calculer les poids . . . 12
3.3 Étape 3 : vérifier les poids . . . 12
3.4 Étape 4 : ajuster le modèle d'analyse pondéré . . . 14

4 Imputation multiple avec mice **16**
4.1 Étape 1 : imputer . . . 16
4.2 Vérifier les imputations . . . 17
4.3 Étapes 2 et 3 : analyser et combiner . . . 18

5 Comparaison des trois approches **21**

Conclusion **24**

Introduction

Bienvenue dans ce TP consacré aux données manquantes ! Vous allez mettre en pratique, sur un vrai jeu de données, les trois approches vues en cours :

1. l'**analyse sur cas complets** ;
2. la **pondération par l'inverse de la probabilité d'être complet** (IPCW) ;
3. l'**imputation multiple** avec le package `mice`.

L'objectif n'est pas de devenir expert en programmation R, mais de voir concrètement comment on réalise chacune de ces analyses, et ce que l'on doit vérifier à chaque étape.

En ligne ou via Rstudio ?

Vous pouvez suivre ce TP en ligne depuis cette page en utilisant les blocs de code interactifs mis à disposition tout au long du TP, ou bien utiliser une session locale de Rstudio. Les données utilisées (`airquality`) sont fournies avec R : il n'y a rien à télécharger. Si vous préférez utiliser Rstudio, il faudra seulement installer les packages nécessaires :

```
install.packages(c("mice", "sandwich", "lmtest"))
```

Si vous faites ce TP en ligne, voici ci-dessous un bloc de code interactif :

Tapez la commande `2+2` ci-dessus. Pour exécuter le code, deux solutions :

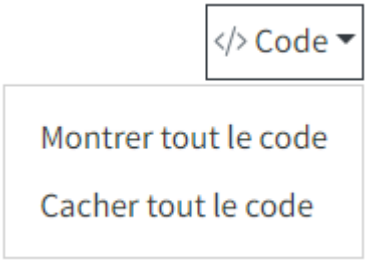
— Cliquer sur 

(cela exécutera l'ensemble du code contenu dans le bloc de code interactif)
— Utiliser le raccourci clavier "Ctrl+Enter" (comme dans Rstudio, ce raccourci exécute le code ligne après ligne quand rien n'est sélectionné en surbrillance, ou alors le code que vous avez préalablement sélectionné en surbrillance avec votre souris).

Solutions

Ce TP peut être réalisé de manière autonome et à votre rythme. A chaque étape, un code de solution est proposé : pour le voir, il suffit de cliquer sur "Montrez moi comment faire !". Il est possible de

démasquer l'ensemble du code de solution d'une page en cliquant sur "</> Code" puis "Montrer



tout le code".

Ce bouton est disponible **en haut de chaque page web**, à droite du titre. Ne restez pas bloqué trop longtemps, et consultez rapidement ces solutions : le but du TP est de comprendre le fonctionnement des méthodes et d'en voir un exemple d'application. Lisez attentivement les tâches à réaliser (écrites en *italique*), inspectez les objets créés par le code de solution, et lisez les commentaires.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

1 Les données et leurs valeurs manquantes

Si vous faites le TD avec Rstudio, les données doivent avoir été préparées avec les commandes suivantes :

```
library(mice)
aq <- airquality                # données fournies avec R
aq$Month <- factor(aq$Month)    # le mois est une variable qualitative
```

1.1 Présentation des données

Les données `airquality` sont des mesures quotidiennes de la qualité de l'air à New York, du 1er mai au 30 septembre 1973. Elles sont fournies avec R, et ont été copiées dans une data.frame `aq` (dans laquelle le mois a été transformé en variable qualitative).

Variable	Description
`Ozone`	Concentration moyenne d'ozone (ppb), de 13h à 15h
`Solar.R`	Rayonnement solaire (Langleys), de 8h à 12h
`Wind`	Vitesse moyenne du vent (mph), de 7h à 10h
`Temp`	Température maximale de la journée (°F)
`Month`	Mois (5 = mai, ..., 9 = septembre)
`Day`	Jour du mois

Question d'intérêt : comment la concentration d'ozone dépend-elle du rayonnement solaire, du vent et de la température ? Nous y répondrons tout au long du TP avec le modèle linéaire suivant :

$$\text{Ozone} = \beta_0 + \beta_1 \, \text{Solar.R} + \beta_2 \, \text{Wind} + \beta_3 \, \text{Temp} + \varepsilon$$

Lancez le code ci-dessous pour avoir un premier aperçu des données.

```
# Vous n'avez rien besoin d'écrire ici, appuyez juste sur le bouton 'Run Code'
# ou sur le raccourci clavier Ctrl+Enter
head(aq)
summary(aq)
```

`summary()` indique le nombre de valeurs manquantes (`NA's`) de chaque variable.

1.2 Combien de données manquantes ?

Comptez le nombre de valeurs manquantes de chaque variable (indice : combinez `is.na()` et `colSums()`).

Correction :

```
colSums(is.na(aq))
## Seules Ozone (37 NA) et Solar.R (7 NA) ont des valeurs manquantes
```

```
  Ozone Solar.R    Wind    Temp   Month     Day
     37       7       0       0       0       0
```

Combien de jours ont toutes leurs variables observées (cas complets) ? Quelle proportion de l'échantillon cela représente-t-il ? (indice : `complete.cases()`)

Correction :

```
sum(complete.cases(aq))   # nombre de cas complets
mean(complete.cases(aq))  # proportion de cas complets
## Une analyse sur cas complets n'utiliserait que 111 jours sur 153 (73 %)
```

```
[1] 111
```

```
[1] 0.7254902
```

La fonction `md.pattern()` du package `mice` résume les **profils de données manquantes** : chaque ligne est un profil (1 = observé, 0 = manquant), le nombre de jours concernés est indiqué à gauche, et le nombre de variables manquantes du profil à droite.

Lancez le code ci-dessous.

```
md.pattern(aq, rotate.names = TRUE)
```

On y lit que 111 jours sont complets, 35 n'ont que `Ozone` manquant, 5 n'ont que `Solar.R` manquant, et 2 ont les deux manquants.

1.3 Le mécanisme est-il MCAR ?

On ne peut jamais prouver que les données ne sont pas MNAR. En revanche, on peut examiner si le fait d'être complet est lié aux variables **observées** : si c'est le cas, l'hypothèse MCAR n'est pas tenable.

Créez une variable `R` dans `aq`, valant 1 si le jour est un cas complet pour les variables du modèle (`Ozone`, `Solar.R`, `Wind`, `Temp`) et 0 sinon. Affichez sa distribution avec `table()`.

Correction :

```
aq$R <- as.numeric(complete.cases(aq[, c("Ozone", "Solar.R", "Wind", "Temp")]))
table(aq$R)
```

```
  0   1
 42 111
```

Calculez la proportion de cas complets selon le mois (indice : `prop.table(table(...), margin = 1)`). Que constatez-vous ?

Correction :

```
prop.table(table(aq$Month, aq$R), margin = 1)
## En juin, seuls 30 % des jours sont complets (contre 74 à 97 % les autres mois) :
## la probabilité d'être complet dépend du mois, qui est observé.
## Les données ne sont donc pas MCAR.
```

```
           0         1
5 0.22580645 0.77419355
6 0.70000000 0.30000000
7 0.16129032 0.83870968
8 0.25806452 0.74193548
9 0.03333333 0.96666667
```

Comparez la température et la vitesse du vent moyennes entre les jours complets et incomplets (indice : `tapply()`).

Correction :

```r
tapply(aq$Temp, aq$R, mean)
tapply(aq$Wind, aq$R, mean)
## Pas de différence notable sur la température et le vent
```

```
       0        1
78.11905 77.79279
```

```
       0        1
10.00476  9.93964
```

> **i** À retenir
>
> La probabilité d'être complet dépend fortement du mois (une panne de l'appareil de mesure de l'ozone en juin est vraisemblable), mais pas de la température ni du vent. Les données ne sont pas MCAR. L'hypothèse MAR (conditionnellement au mois) est plausible, mais elle n'est **pas vérifiable** : on ne peut pas exclure que la non-réponse dépende aussi des valeurs d'ozone elles-mêmes.

Conseils importants avant de passer à la suite

- *Ne restez pas bloqué longtemps.* Un bouton "Montrez moi comment faire !" est *toujours* disponible, n'hésitez pas à l'utiliser après 5 min de galère : le but est de voir en pratique les notions abordées en cours, pas de devenir programmeur R.
- *De l'aide est disponible :*
  - Taper `?md.pattern` vous affichera la page d'aide correspondante, comme vous le faites habituellement dans Rstudio.
  - Le jour du TP, lever la main téléportera automatiquement l'enseignant auprès de vous.
- *Il n'y a pas toujours qu'une seule manière de faire.* La correction proposée à chaque question n'est qu'une proposition.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

2 Analyse sur cas complets

Si vous faites le TD avec Rstudio, les données doivent avoir été préparées avec les commandes suivantes :

```
aq <- airquality
aq$Month <- factor(aq$Month)
aq$R <- as.numeric(complete.cases(aq[, c("Ozone", "Solar.R", "Wind", "Temp")]))
```

L'analyse sur cas complets est l'approche par défaut de la plupart des logiciels : les observations ayant au moins une valeur manquante sur les variables du modèle sont **silencieusement exclues**. C'est le comportement de la fonction `lm()` (option `na.action = na.omit` par défaut).

Ajustez le modèle linéaire expliquant ***`Ozone`*** *par* ***`Solar.R`****,* ***`Wind`*** *et* ***`Temp`****, stockez-le dans un objet* ***`fit_cc`****, et affichez ses résultats avec* ***`summary()`****.*

Correction :

```
fit_cc <- lm(Ozone ~ Solar.R + Wind + Temp, data = aq)
summary(fit_cc)
## Remarquez la mention "(42 observations deleted due to missingness)"
```

```
Call:
lm(formula = Ozone ~ Solar.R + Wind + Temp, data = aq)

Residuals:
    Min      1Q  Median      3Q     Max
-40.485 -14.219  -3.551  10.097  95.619

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -64.34208   23.05472  -2.791  0.00623 **
Solar.R       0.05982    0.02319   2.580  0.01124 *
Wind         -3.33359    0.65441  -5.094 1.52e-06 ***
Temp          1.65209    0.25353   6.516 2.42e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 21.18 on 107 degrees of freedom
```

```
  (42 observations effacées parce que manquantes)
Multiple R-squared:  0.6059,    Adjusted R-squared:  0.5948
F-statistic: 54.83 on 3 and 107 DF,  p-value: < 2.2e-16
```

Combien d'observations ont réellement été utilisées pour ajuster le modèle ? (indice : `nobs()`)

Correction :

```
nobs(fit_cc)
## 111 jours sur 153 : on a perdu 27 % de l'échantillon, y compris les 5 jours
## où l'ozone était observé mais pas le rayonnement solaire
```

```
[1] 111
```

Affichez les intervalles de confiance à 95 % des coefficients (indice : `confint()`).

Correction :

```
confint(fit_cc)
```

```
                    2.5 %      97.5 %
(Intercept) -110.04538108 -18.6387768
Solar.R        0.01385613   0.1057851
Wind          -4.63087706  -2.0363055
Temp           1.14949967   2.1546862
```

> **i** À retenir
>
> — L'analyse sur cas complets est simple, mais elle **perd de l'information** : ici, 42 jours sont exclus, dont certains ne sont incomplets que sur une seule variable.
> — Elle est sans biais si les données sont MCAR, ou plus généralement si la probabilité d'être complet ne dépend pas de la variable à expliquer (`Ozone`) conditionnellement aux variables explicatives du modèle.
> — Ici, la probabilité d'être complet dépend du mois, qui n'est pas dans le modèle. Si le mois est lié à l'ozone au-delà de ce qu'expliquent le rayonnement, le vent et la température, l'analyse sur cas complets peut être biaisée.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

3 Pondération IPCW

Si vous faites le TD avec Rstudio, les données doivent avoir été préparées avec les commandes suivantes :

```
library(sandwich)
library(lmtest)
aq <- airquality
aq$Month <- factor(aq$Month)
aq$R <- as.numeric(complete.cases(aq[, c("Ozone", "Solar.R", "Wind", "Temp")]))
fit_cc <- lm(Ozone ~ Solar.R + Wind + Temp, data = aq)
```

L'IPCW consiste à pondérer chaque cas complet par l'inverse de sa probabilité d'être complet, $w_i = 1/\hat{\pi}_i$, où $\hat{\pi}_i = \widehat{\Pr}(R_i = 1 \mid H_i)$. Les jours complets qui « ressemblent » à des jours incomplets reçoivent un poids élevé et les représentent dans l'analyse.

3.1 Étape 1 : modéliser la probabilité d'être complet

Les variables H du modèle de pondération doivent être **complètement observées**. Ici, `Ozone` et `Solar.R` ne peuvent donc pas en faire partie : on utilise `Wind`, `Temp` et `Month`.

Ajustez un modèle de régression logistique expliquant `R` par `Wind`, `Temp` et `Month` sur l'ensemble des 153 jours. Stockez-le dans un objet `mod_R` et affichez ses résultats.

Correction :

```
mod_R <- glm(R ~ Wind + Temp + Month, data = aq, family = binomial)
summary(mod_R)
## Le modèle est ajusté sur tous les jours (R, Wind, Temp et Month sont
## observés pour tous). Seul le mois de juin est fortement associé à R.
```

```
Call:
glm(formula = R ~ Wind + Temp + Month, family = binomial, data = aq)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.38676    2.70700   0.143  0.88639
Wind         0.01412    0.06604   0.214  0.83071
```

```
Temp         0.01041    0.03548   0.294  0.76911
Month6      -2.20402    0.75268  -2.928  0.00341 **
Month7       0.26279    0.89772   0.293  0.76973
Month8      -0.32843    0.85791  -0.383  0.70185
Month9       2.03829    1.16563   1.749  0.08035 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 179.83  on 152  degrees of freedom
Residual deviance: 141.23  on 146  degrees of freedom
AIC: 155.23

Number of Fisher Scoring iterations: 6
```

3.2 Étape 2 : calculer les poids

Calculez pour chaque jour la probabilité estimée d'être complet (indice : `predict(..., type = "response")`), stockez-la dans `aq$p`, puis calculez les poids `aq$w = 1/aq$p`.

Correction :

```r
aq$p <- predict(mod_R, type = "response")
aq$w <- 1 / aq$p
head(aq[, c("Month", "R", "p", "w")])
```

```
  Month R         p        w
1     5 1 0.7665538 1.304540
2     5 1 0.7772120 1.286650
3     5 1 0.7917098 1.263089
4     5 1 0.7675928 1.302774
5     5 0 0.7634732 1.309804
6     5 0 0.7832019 1.276810
```

3.3 Étape 3 : vérifier les poids

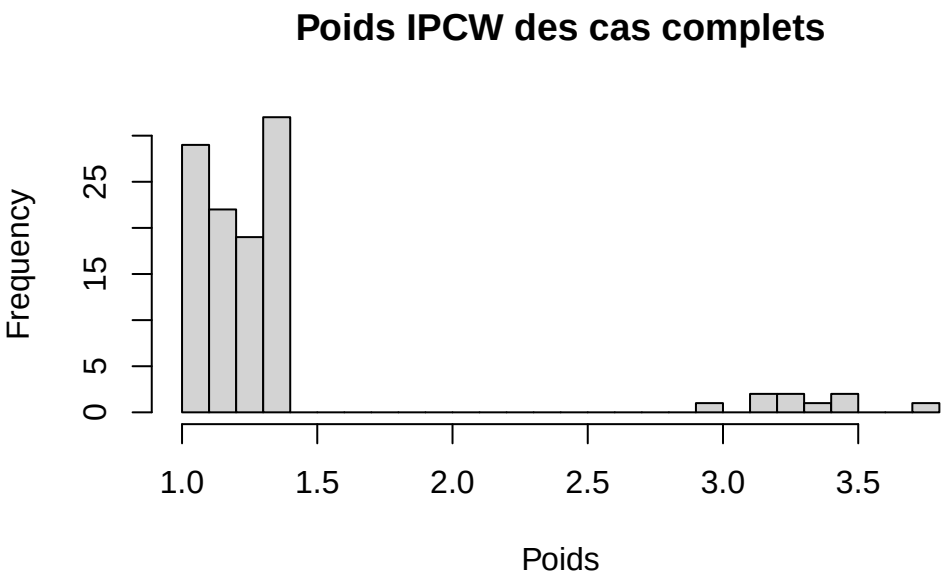
Avant d'utiliser les poids, il faut s'assurer qu'ils ne sont pas extrêmes : quelques jours avec des poids très élevés domineraient l'analyse.

Décrivez la distribution des poids chez les cas complets (`summary()` et `hist()`), puis calculez la taille d'échantillon effective $n_{eff} = (\sum w_i)^2 / \sum w_i^2$.

Correction :

```
w_cc <- aq$w[aq$R == 1]   # poids des cas complets
summary(w_cc)
hist(w_cc, breaks = 20, main = "Poids IPCW des cas complets", xlab = "Poids")
sum(w_cc)^2 / sum(w_cc^2) # taille d'échantillon effective
## Le poids maximal est inférieur à 4 (ce sont les jours complets de juin) :
## pas de poids extrême. La taille effective (~94) est assez proche du
## nombre de cas complets (111).
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.030   1.038   1.237   1.376   1.326   3.747
```




```
[1] 93.88585
```

On vérifie aussi que la pondération remplit son rôle : une fois pondérés, les cas complets doivent avoir la même répartition sur H que l'échantillon entier.

Comparez la répartition des mois dans l'échantillon entier, chez les cas complets non pondérés, et chez les cas complets pondérés. Le code ci-dessous n'a pas besoin d'être modifié (il utilise les poids ***`aq$w`*** *calculés à l'étape 2).*

```
cc <- aq$R == 1
round(cbind(
  echantillon = prop.table(table(aq$Month)),
  cas_complets = prop.table(table(aq$Month[cc])),
  cas_complets_ponderes = tapply(aq$w[cc], aq$Month[cc], sum) / sum(aq$w[cc])
), 3)
```

```
  echantillon cas_complets cas_complets_ponderes
5       0.203        0.216                 0.203
6       0.196        0.081                 0.195
7       0.203        0.234                 0.203
8       0.203        0.207                 0.203
9       0.196        0.261                 0.196
```

Le mois de juin, sous-représenté chez les cas complets (8 % au lieu de 20 %), retrouve sa place une fois les poids appliqués.

3.4 Étape 4 : ajuster le modèle d'analyse pondéré

Le modèle d'analyse est ajusté sur les cas complets en utilisant les poids. Avec la pondération, certains jours comptent plus que d'autres : un jour complet de juin « représente » à lui seul près de 4 jours. Mais l'échantillon n'a pas réellement grossi : ce jour reste une seule mesure. Les erreurs standard usuelles de `lm()` ne sont donc pas adaptées. On utilise une **variance robuste** (« sandwich »), qui estime la variabilité à partir des données réellement observées, avec les packages `sandwich` et `lmtest`.

Ajustez le modèle linéaire pondéré (option `weights = w` de `lm()`) et stockez-le dans `fit_ipcw`. Affichez ensuite les coefficients avec des erreurs standard robustes : `coeftest(fit_ipcw, vcov = vcovHC(fit_ipcw, type = "HC0"))`.

Correction :

```r
fit_ipcw <- lm(Ozone ~ Solar.R + Wind + Temp, data = aq, weights = w)
## lm() exclut toujours les jours incomplets : seuls les cas complets,
## pondérés, contribuent à l'estimation
coeftest(fit_ipcw, vcov = vcovHC(fit_ipcw, type = "HC0"))
```

```
t test of coefficients:

             Estimate Std. Error t value  Pr(>|t|)
(Intercept) -65.530361  22.047557 -2.9722  0.003652 **
Solar.R       0.070876   0.021413  3.3100  0.001272 **
Wind         -3.073827   0.950693 -3.2332  0.001628 **
Temp          1.602989   0.208871  7.6745 8.214e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Comparez les coefficients obtenus avec ceux de l'analyse sur cas complets (`coef(fit_cc)`).

Correction :

```
round(cbind(cas_complets = coef(fit_cc), ipcw = coef(fit_ipcw)), 3)
## Les estimations sont proches : le mois, qui explique la non-réponse,
## semble peu lié à l'ozone une fois pris en compte le rayonnement,
## le vent et la température.
```

```
            cas_complets    ipcw
(Intercept)      -64.342 -65.530
Solar.R            0.060   0.071
Wind              -3.334  -3.074
Temp               1.652   1.603
```

> **À retenir**
>
> — L'IPCW n'utilise, comme l'analyse sur cas complets, que les jours complets : les 5 jours où seul `Solar.R` manque sont toujours perdus.
> — Les variables du modèle de pondération doivent être complètement observées : la variable à expliquer `Ozone`, incomplète, ne peut pas y figurer.
> — Il faut toujours vérifier les poids (valeurs extrêmes, taille effective, équilibre sur H) et utiliser une variance robuste (ici `coeftest()` avec `vcovHC()`, étape 4).

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

4 Imputation multiple avec mice

Si vous faites le TD avec Rstudio, les données doivent avoir été préparées avec les commandes suivantes :

```
library(mice)
aq <- airquality
aq$Month <- factor(aq$Month)
```

L'imputation multiple se déroule en trois étapes : (1) **imputer** m fois les valeurs manquantes, (2) **analyser** chacun des m jeux de données complétés, (3) **combiner** les m résultats avec les règles de Rubin.

4.1 Étape 1 : imputer

On choisit d'abord les variables du modèle d'imputation. La règle est d'être inclusif : toutes les variables du modèle d'analyse (**y compris la variable à expliquer `Ozone`**, même si elle est incomplète), ainsi que les variables auxiliaires liées à la non-réponse ou aux variables incomplètes, comme le mois.

Par défaut, `mice()` utilise **toutes les colonnes** de la data.frame qu'on lui donne comme prédicteurs dans le modèle d'imputation. Il faut donc lui fournir uniquement les variables retenues : ici, on exclut `Day` (le jour du mois, de 1 à 31), qui n'apporte pas d'information utile pour prédire les valeurs manquantes.

Créez une data.frame `dat` contenant les variables `Ozone`, `Solar.R`, `Wind`, `Temp` et `Month`.

Correction :

```
dat <- aq[, c("Ozone", "Solar.R", "Wind", "Temp", "Month")]
## Day est exclu : sinon mice l'utiliserait comme prédicteur
```

Lancez l'imputation avec `mice()` : 20 imputations (`m = 20`), une graine pour la reproductibilité (`seed = 123`), et `printFlag = FALSE` pour ne pas afficher le détail des itérations. Stockez le résultat dans un objet `imp`, puis affichez `imp$method`.

Correction :

```r
imp <- mice(dat, m = 20, seed = 123, printFlag = FALSE)
imp$method
## Par défaut, les variables continues sont imputées par Predictive Mean
## Matching ("pmm"). C'est adapté ici : l'ozone est positif et sa distribution
## est asymétrique, et pmm n'impute que des valeurs déjà observées.
## Les variables complètes ("") ne sont pas imputées.
```

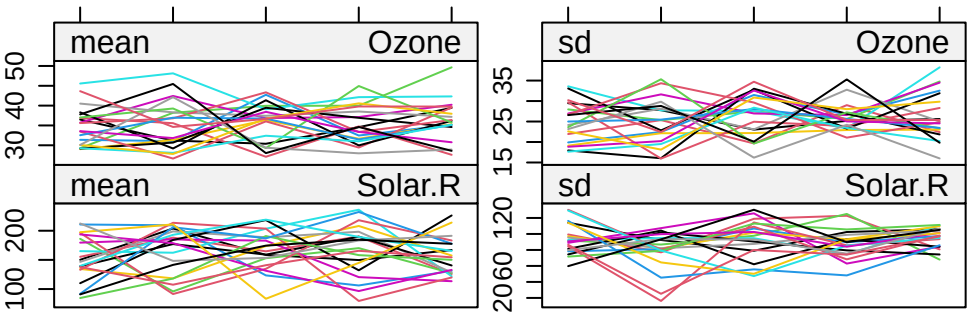
```
  Ozone Solar.R    Wind    Temp   Month
  "pmm"   "pmm"      ""      ""      ""
```

4.2 Vérifier les imputations

Vérifiez la convergence de l'algorithme avec ***`plot(imp)`****. On attend des courbes qui se mélangent, sans tendance.*

Correction :

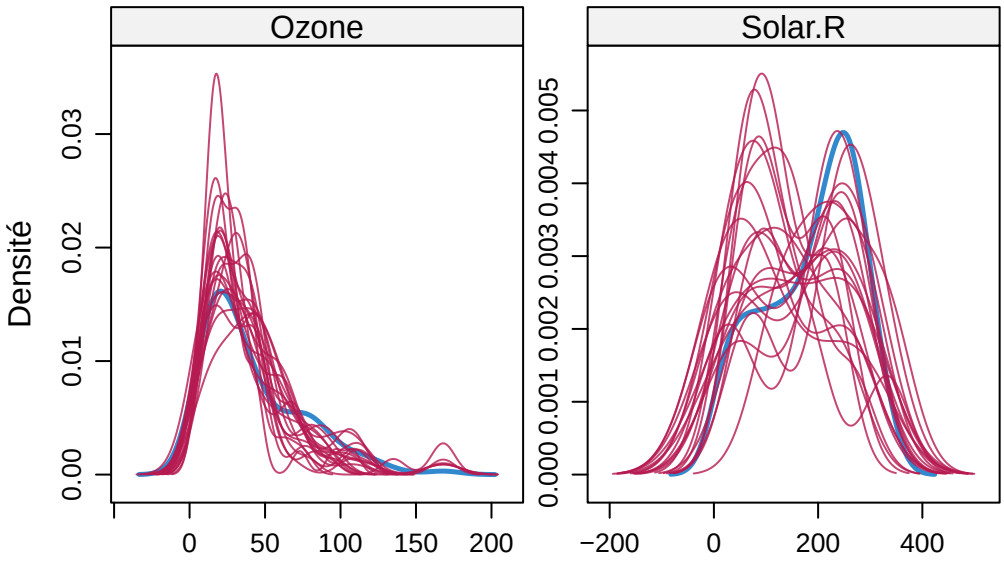
```r
plot(imp)
## Moyenne et écart-type des valeurs imputées au fil des itérations,
## pour chacune des 20 imputations : pas de tendance, convergence correcte.
```



Comparez la distribution des valeurs observées (en bleu) et imputées (en rouge) avec ***`densityplot(imp)`****.*

Correction :

```r
densityplot(imp)
## Les distributions imputées sont plausibles. Elles n'ont pas à être
## identiques aux distributions observées : sous MAR, les jours incomplets
## peuvent différer des jours complets.
```



4.3 Étapes 2 et 3 : analyser et combiner

Ajustez le modèle linéaire sur chacun des 20 jeux imputés avec `with(imp, lm(...))`, stockez le résultat dans `fit_mi`, puis combinez les résultats avec `pool()` et affichez-les avec `summary(..., conf.int = TRUE)`.

Correction :

```r
fit_mi <- with(imp, lm(Ozone ~ Solar.R + Wind + Temp))
pooled <- pool(fit_mi)
summary(pooled, conf.int = TRUE)
## Ces estimations utilisent les 153 jours, y compris ceux qui n'ont qu'une
## seule variable manquante
```

```
         term    estimate  std.error statistic        df      p.value
1 (Intercept) -67.2396776 23.8385880 -2.820623  63.15159 6.397662e-03
2     Solar.R   0.0587741  0.0220801  2.661858 103.98668 9.005027e-03
3        Wind  -2.8309067  0.6400799 -4.422739  85.74650 2.844500e-05
4        Temp   1.6087174  0.2671457  6.021873  60.48493 1.102030e-07
          2.5 %      97.5 %      conf.low    conf.high
1 -114.87505676 -19.6042984 -114.87505676 -19.6042984
```

```
2    0.01498837   0.1025598    0.01498837   0.1025598
3   -4.10339712  -1.5584163   -4.10339712  -1.5584163
4    1.07443455   2.1430002    1.07443455   2.1430002
```

Les règles de Rubin décomposent la variance totale T de chaque coefficient en deux parties :

$$T = \bar{U} + \left(1 + \tfrac{1}{m}\right) B$$

- $\bar{U}$ (colonne `ubar`) est la moyenne des variances obtenues dans chaque jeu imputé : c'est la variance qu'on aurait si les données étaient complètes ;
- B (colonne `b`) est la variance des estimations **entre** les jeux imputés : elle traduit l'incertitude sur les valeurs imputées, donc les données manquantes.

La **fraction d'information manquante** (`fmi`) est, à peu de chose près, la part de T qui vient de B. Elle s'interprète comme la proportion d'information sur le coefficient perdue à cause des données manquantes : avec `fmi` = 0,30, la variance de l'estimation serait environ 30 % plus faible si les données étaient complètes.

Affichez les colonnes `term`, `ubar`, `b`, `t` et `fmi` de `pooled$pooled`.

Correction :

```r
pooled$pooled[, c("term", "ubar", "b", "t", "fmi")]
## Selon les coefficients, environ 20 à 35 % de l'information est perdue
## à cause des données manquantes
```

```
         term         ubar            b            t       fmi
1 (Intercept) 3.812855e+02 1.780884e+02 5.682783e+02 0.3493366
2     Solar.R 4.063907e-04 7.727630e-05 4.875308e-04 0.1820134
3        Wind 3.149468e-01 9.024332e-02 4.097023e-01 0.2486029
4        Temp 4.690237e-02 2.329946e-02 7.136681e-02 0.3635027
```

Une fraction d'information manquante élevée ?

Perdre 20 à 35 % de l'information peut surprendre, mais c'est cohérent avec les données : l'ozone, la **variable à expliquer**, manque pour 24 % des jours. Or un jour où l'ozone manque n'apporte presque rien à l'estimation des coefficients : sa valeur imputée est prédite à partir des mêmes variables que celles du modèle d'analyse. L'information perdue est donc du même ordre que la proportion de jours où l'ozone manque.
La `fmi` peut être plus faible que la proportion de données manquantes lorsque des variables auxiliaires prédisent bien les valeurs manquantes. C'est l'intérêt d'un modèle d'imputation riche.
Enfin, la `fmi` est elle-même estimée de manière imprécise : relancez `mice()` avec une autre graine (`seed`), et les valeurs pourront varier. Augmenter m la rend plus stable.

À retenir

- Contrairement à l'IPCW, l'imputation multiple utilise toutes les observations, y compris celles qui sont partiellement observées, et la variable à expliquer **doit** figurer dans le modèle d'imputation.
- Il faut vérifier la convergence et la plausibilité des valeurs imputées.
- Les règles de Rubin intègrent dans la variance l'incertitude due aux données manquantes (variance entre imputations).

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

5 Comparaison des trois approches

Si vous faites le TD avec Rstudio, les objets des parties précédentes doivent avoir été créés avec les commandes suivantes :

```
library(mice)
library(sandwich)
library(lmtest)
aq <- airquality
aq$Month <- factor(aq$Month)
aq$R <- as.numeric(complete.cases(aq[, c("Ozone", "Solar.R", "Wind", "Temp")]))
## Cas complets
fit_cc <- lm(Ozone ~ Solar.R + Wind + Temp, data = aq)
## IPCW
mod_R <- glm(R ~ Wind + Temp + Month, data = aq, family = binomial)
aq$w <- 1 / predict(mod_R, type = "response")
fit_ipcw <- lm(Ozone ~ Solar.R + Wind + Temp, data = aq, weights = w)
## Imputation multiple
imp <- mice(aq[, c("Ozone", "Solar.R", "Wind", "Temp", "Month")],
            m = 20, seed = 123, printFlag = FALSE)
pooled <- pool(with(imp, lm(Ozone ~ Solar.R + Wind + Temp)))
```

Les trois modèles des parties précédentes sont déjà disponibles : `fit_cc` (cas complets), `fit_ipcw` (IPCW) et `pooled` (imputation multiple).

Lancez le code ci-dessous, qui rassemble les coefficients et leurs erreurs standard dans un même tableau.

```
se_ipcw <- sqrt(diag(vcovHC(fit_ipcw, type = "HC0")))  # erreurs standard robustes
res_mi <- summary(pooled)

round(data.frame(
  coef_cc   = coef(fit_cc),
  se_cc     = coef(summary(fit_cc))[, "Std. Error"],
  coef_ipcw = coef(fit_ipcw),
  se_ipcw   = se_ipcw,
  coef_mi   = res_mi$estimate,
  se_mi     = res_mi$std.error
), 3)
```

```
            coef_cc  se_cc coef_ipcw se_ipcw coef_mi  se_mi
(Intercept) -64.342 23.055   -65.530  22.048 -67.240 23.839
Solar.R       0.060  0.023     0.071   0.021   0.059  0.022
```

```
Wind         -3.334  0.654    -3.074   0.951  -2.831  0.640
Temp          1.652  0.254     1.603   0.209   1.609  0.267
```

Répondez aux questions suivantes avant de dérouler les réponses.

1. Les trois approches donnent-elles des conclusions différentes sur l'effet du rayonnement solaire, du vent et de la température ?

Réponse

Non : les coefficients sont proches et de même signe, et les trois effets restent nettement différents de 0. La non-réponse dépend surtout du mois, qui semble peu lié à l'ozone une fois pris en compte le rayonnement, le vent et la température. Dans cet exemple, l'analyse sur cas complets n'est donc probablement pas très biaisée. C'est une conclusion propre à ces données : le cours a montré par simulation des situations où les écarts sont importants.

2. Pourquoi l'erreur standard du coefficient de `Wind` est-elle plus grande avec l'IPCW ?

Réponse

Après une pondération IPCW, on utilise une variance robuste. Avec les poids, certains jours comptent plus que d'autres dans l'estimation, mais l'échantillon n'a pas réellement grossi : un jour de juin qui « représente » 4 jours reste une seule mesure. La variance robuste estime la variabilité à partir des données réellement observées, sans faire comme si les poids avaient augmenté l'effectif.
Le résultat est souvent moins précis qu'avec l'analyse sur cas complets : c'est le prix à payer pour corriger le biais de sélection.

3. L'imputation multiple utilise 153 jours au lieu de 111. Pourquoi ses erreurs standard ne sont-elles pas beaucoup plus petites que celles de l'analyse sur cas complets ?

Réponse

La plupart des jours incomplets (37 sur 42) ont la variable à expliquer `Ozone` manquante. Les valeurs d'ozone imputées sont prédites à partir des variables explicatives du modèle : elles apportent peu d'information nouvelle sur les coefficients, et la variance entre imputations en tient compte. Seuls les 5 jours où l'ozone est observé et `Solar.R` manquant apportent réellement de l'information supplémentaire. Le gain de l'imputation multiple est plus net quand ce sont les variables explicatives qui sont incomplètes.

4. Sur quelle hypothèse reposent ces trois analyses, et peut-on la vérifier ?

Réponse

L'analyse sur cas complets suppose que la non-réponse ne dépend pas de l'ozone conditionnellement aux variables explicatives ; l'IPCW et l'imputation multiple supposent que les données sont MAR conditionnellement aux variables utilisées (mois, vent, température…). Aucune de ces hypothèses n'est vérifiable à partir des données : si la panne de l'appareil était liée aux

pics d'ozone eux-mêmes (MNAR), aucune des trois méthodes ne corrigerait le biais. Seule une analyse de sensibilité permettrait d'évaluer la robustesse des conclusions.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

Conclusion

Bravo, vous avez terminé ce TP consacré aux données manquantes !

Pour toute analyse avec des données manquantes, gardez en tête la démarche suivie :

1. **Décrire** les données manquantes : combien, sur quelles variables, selon quels profils.
2. **Explorer** le lien entre la non-réponse et les variables observées, pour juger de la plausibilité des hypothèses MCAR et MAR (sans oublier que l'hypothèse MNAR ne peut jamais être exclue).
3. **Choisir** une méthode adaptée : analyse sur cas complets, IPCW ou imputation multiple, en vérifiant à chaque fois les points de contrôle (poids, convergence, plausibilité des imputations).
4. **Confronter** les résultats des différentes approches, et compléter si besoin par une analyse de sensibilité.

Pour aller plus loin, le livre de Stef van Buuren, auteur du package `mice`, est disponible gratuitement en ligne : Flexible Imputation of Missing Data.